

Why Linear Algebra?

Linear algebra is a branch of mathematics that deals with **vectors, matrices, linear systems, and linear transformations**.

It involves understanding and manipulating multi-dimensional data through concepts like vector spaces, matrix operations, and matrix factorizations.

Example:

Student Height GPA Hours Studied

Alice 170 3.8 18

Bob 182 3.2 8

Carol 160 3.9 22

How would a computer store this?

$$X = \begin{bmatrix} 170 & 3.8 & 18 \\ 182 & 3.2 & 8 \\ 160 & 3.9 & 22 \end{bmatrix}$$

Every machine learning Algorithm begins with a matrix!

How would you represent Alice here?

Example: PCA

Suppose you have

- 5000 genes
- 100 patients

Can we visualize this?

No.

Linear algebra gives us **PCA**.

PCA finds new axes where the data varies the most.

Suddenly 5000 dimensions become 2!

One key question we will try to answer in this course:

How can we do matrix computations with acceptable speed and acceptable accuracy?

Three modes of thinking:

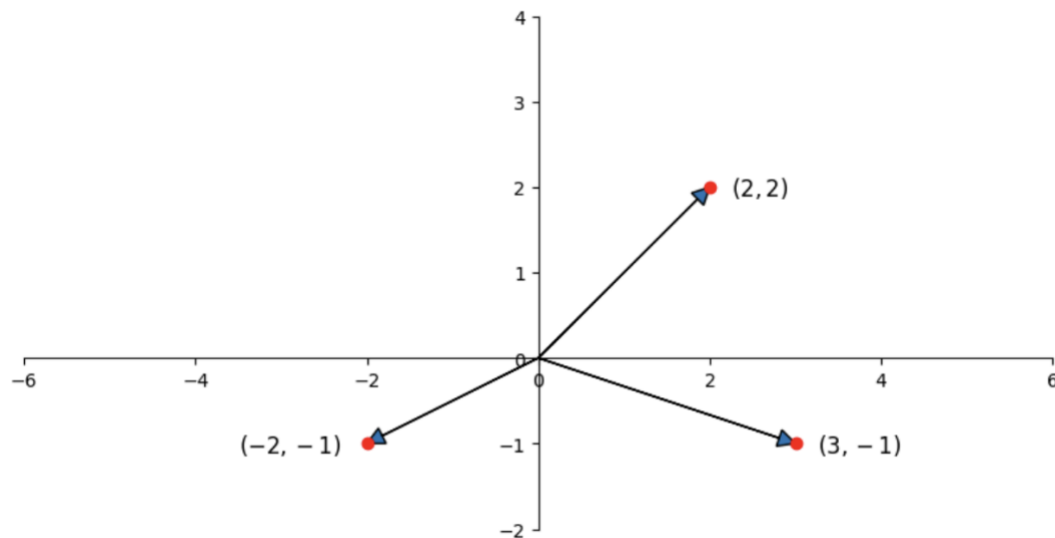
- **Algebraic thinking:** how to correctly manipulate symbols in a consistent logical framework, for example to solve equations.
- **Geometric thinking:** learning to extend familiar two- and three-dimensional concepts to higher dimensions.
- **Computational thinking:** understanding the relationship between abstract algebraic machinery and actual computations which efficiently arrive at the correct answer.

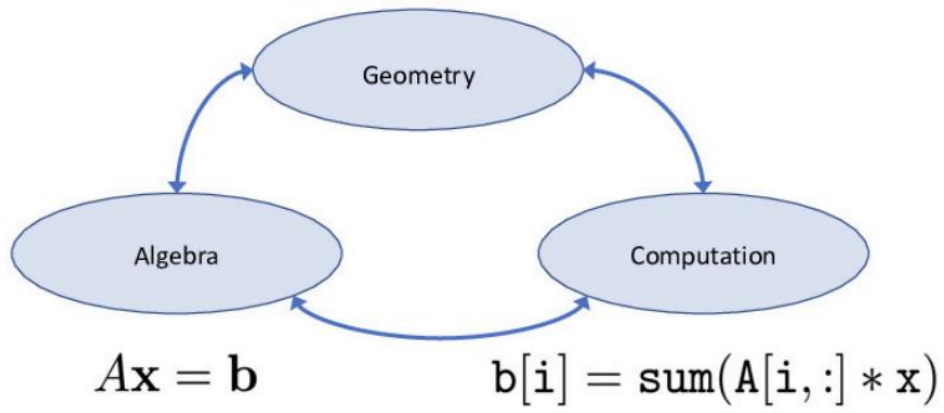
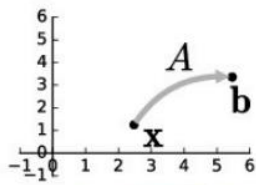
Example. We can view a vector $\mathbf{v}$ using all three modes of thinking:

- **Algebraically:** a vector $\mathbf{v}$ is an abstract object that permits addition $\mathbf{v} + \mathbf{w}$ and multiplication by a constant or scalar c to form $c\mathbf{v}$. It's that simple: any collection of objects that you can add and scalar-multiply can form a vector space.
- **Computationally:** a vector is an ordered list of real numbers. Here are a few examples in $\mathbb{R}^2$, meaning that the list contains two elements and each one is a real number:

$$\mathbf{v} = \begin{bmatrix} 3 \\ 1 \end{bmatrix}, \mathbf{u} = \begin{bmatrix} 2 \\ 2 \end{bmatrix}, \mathbf{w} = \begin{bmatrix} -2 \\ -1 \end{bmatrix}$$

- **Geometrically:** a vector is a point in n -dimensional space. Here's how we can represent the three vectors from above:





Linear Algebra for Data Science

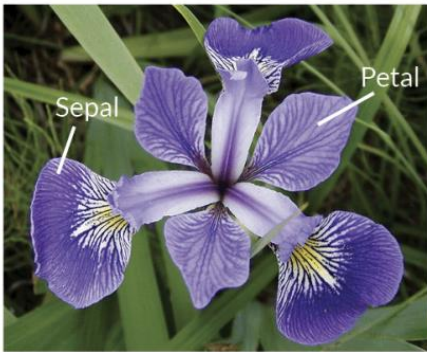
Let us consider the well-known Iris Dataset.

This dataset contains 150 rows.

Each row contains 4 columns that contain the following features:

['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']

Specifically, it contains **50 samples** of each of **three species** of irises, and **4 attributes** about each iris sample.



Iris Versicolor



Iris Setosa



Iris Virginica

This data can be represented as a matrix:

and

```
1 A = iris.data
2 print(A.shape)
```

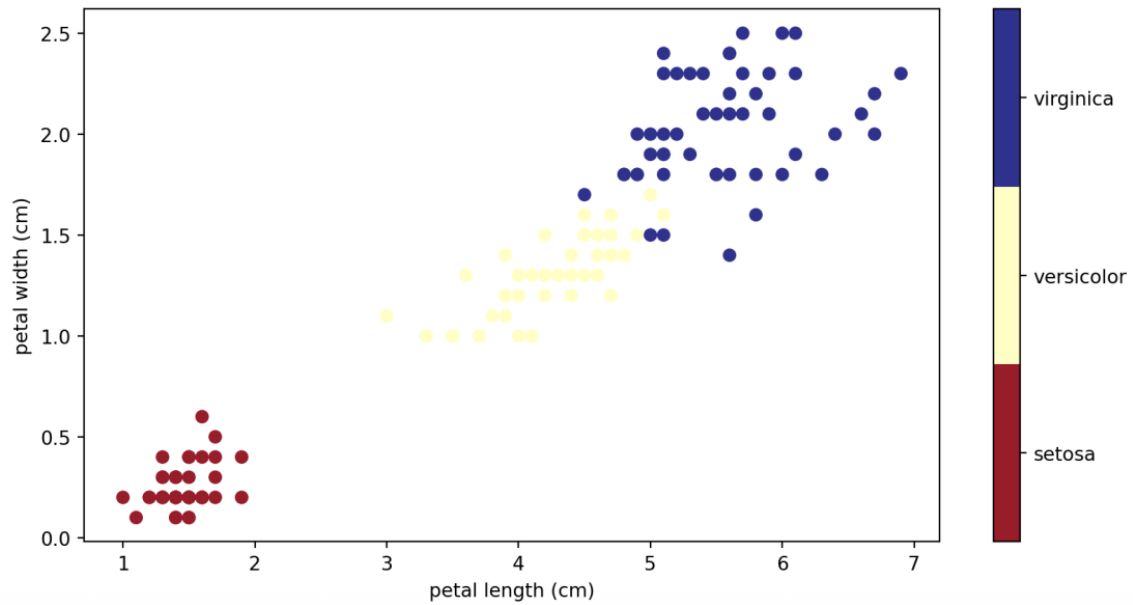
```
(150, 4)
```

```
1 print(A)
```

```
[[5.1 3.5 1.4 0.2]
 [4.9 3.  1.4 0.2]
 [4.7 3.2 1.3 0.2]
 [4.6 3.1 1.5 0.2]
 [5.  3.6 1.4 0.2]
 [5.4 3.9 1.7 0.4]
 [4.6 3.4 1.4 0.3]
 [5.  3.4 1.5 0.2]
 [4.4 2.9 1.4 0.2]
 [4.9 3.1 1.5 0.1]
 [5.4 3.7 1.5 0.2]
 [4.8 3.4 1.6 0.2]
 [4.8 3.  1.4 0.1]
 [4.3 3.  1.1 0.1]
 ...]
```

And of course there are going to be many more such rows!

This dataset might look random to the human eye. However, if we plot its attributes, we can see that there is structure within the data.



Observations:

1. There are *correlations* among the 150 rows of A .

- Knowing the petal length of an iris tells you something about the petal width.
- Knowing the petal length and width might give you a way to classify which type of iris you have.

2. The matrix is *compressible*.

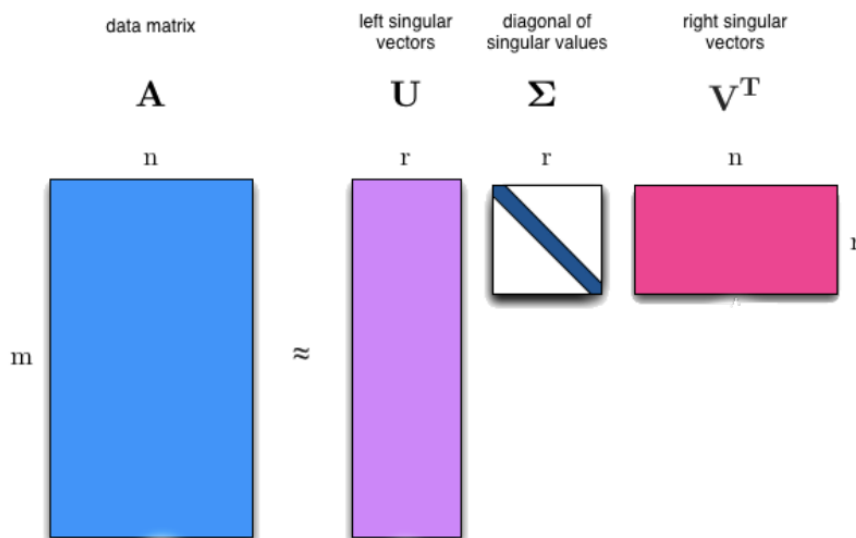
- The 150 rows of are all samples of the same underlying physical process.
- We should be able to decompose this matrix into simpler matrices.

To do this, we will explore several properties of matrices that might make them "simpler" than other ones:

- **Size:** one matrix might have fewer rows or columns than another, which means it is more compact and has fewer degrees of freedom.
- **Sparsity:** a matrix might be lower triangular (denoted L) or upper triangular (denoted U or R), with all other entries equal to zero.
- **Symmetry:** the entries of the matrix might be symmetric across the diagonal S , or it might even be a diagonal matrix Σ with 0s on all other entries.

In this course, we will learn several algorithms for matrix factorization or decomposition:

- $A = LU$: Solves system of Linear equations
- $A = QR$: Gram - Schmidt Orthogonalization
- $A = U\Sigma V^T$: Singular Value Decomposition on SVD



What's amazing is that you can factor (essentially) every matrix in this way!

This is linear algebra's version of compression: the large iris dataset with its m rows and n attributes actually only has r independent dimensions.

Word of Caution

Many of the techniques in this course involve dealing with data sets that are high-dimensional but where we believe the data has low intrinsic dimensionality. This is often a good assumption, but not always!

If the data are in fact independent samples of a high-dimensional distribution, then our machine learning algorithms will seek in vain to find structure where there isn't any. In this case, an exact decomposition will be correct but unhelpful in simplifying the data (since it isn't possible), and an approximate decomposition will simplify the data (because we forced it) but its resulting predictions might be very, very wrong!